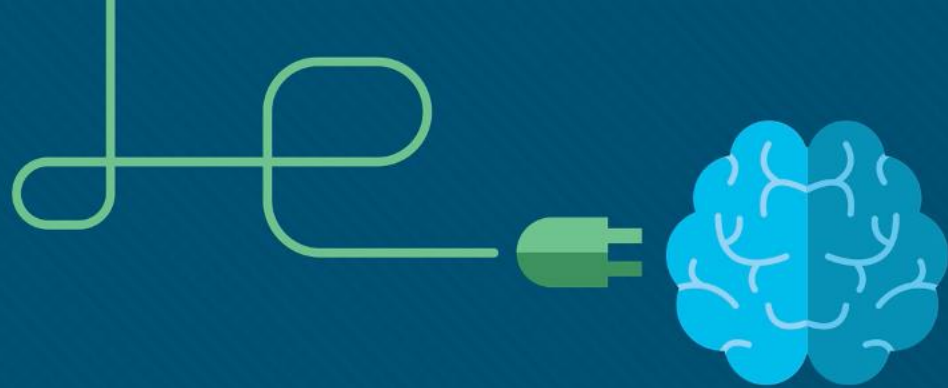




Module 1: Getting Started with HTML

Introduction to HTML



Module Objectives

Module Title: Introduction to HTML

Module Objective: In this module, you'll be introduced to the basics of HTML, including its history, structure, and syntax. You'll learn how to set up an HTML development environment, use character encoding, and understand the key elements, attributes, and tags that form the foundation of HTML. You'll also explore how HTML interacts with CSS and JavaScript to create dynamic and visually appealing web pages.

Skills you'll gain: Basic HTML structure, understanding of key tags and attributes, ability to set up a development environment, and foundational knowledge of web development.

At the end of this module, you'll complete a project to apply what you've learned and a test to wrap up your understanding.

Module Objectives

Course Syllabus:

The course is organized into six modules, each focusing on key aspects of HTML, along with a final project and test to solidify your learning. Below is a detailed breakdown of the syllabus:

Module 1: Getting Started with HTML

- Introduction to HTML
- The Birth, Evolution, and Future of HTML
- HTML Development Environment
- Character Encoding
- HTML Syntax: Elements, Tags, and Comments
- HTML Syntax: Attributes and Values
- HTML Syntax: HTML, Head, and Body Tags
- The Relationship Between HTML, CSS, and JS
- Module 1 Completion

1.1 What is HTML?

1.1.1 What is HTML?

HTML is like the skeleton of a website, giving it structure and form. It sets up everything you see online, from text and images to forms and buttons. Without HTML, websites would just be plain text without any organization.

HTML = HyperText Markup Language

- ***Hypertext*** refers to text that contains links to other documents or resources. In HTML, hypertext allows users to navigate between different web pages by clicking on links.
- ***Markup*** refers to the annotations or tags that are used to define the structure and formatting of content in a document. In HTML, markup tags are used to specify elements such as headings, paragraphs, images, and links that make a web page visually appealing and easy to navigate. These tags provide information to web browsers about how the content should be structured, styled, and rendered on the user's screen.
- ***Language*** refers to the system of rules and syntax used to write and interpret the markup in a consistent and standardized manner. HTML provides a set of predefined tags and attributes that define how content should be displayed and interacted with in a web browser. It's the universal translator, the language that humans, computers and browsers understand.

1.1.1 What is HTML?



HTML's Best Friends: CSS and JavaScript

With HTML you have the power to organize and structure your content in a way that makes sense. It's like creating a blueprint for your web page, where you can define headings, paragraphs, images, links, and so much more. Think of it as your elementary tool for crafting engaging and interactive online experiences.

As you will notice soon, HTML is not alone in this adventure. It teams up with its partners in crime, CSS (Cascading Style Sheets) and JavaScript. Think of CSS as the paint and decorations that make a room look good, giving websites their style and flair. JavaScript is like the gadgets in a room, adding fun and interactive features that make the website more interesting and useful.

1.2 The Birth, Evolution, and Future of HTML

1.2 The Birth, Evolution, and Future of HTML

It all started back in 1989 when Sir **Tim Berners-Lee** invented HTML while working at CERN, the European Organization for Nuclear Research. His vision was to create a way to share scientific information easily across different computers.

The first web page, created by Sir Tim Berners-Lee, was a simple yet groundbreaking creation. It consisted of plain text and hyperlinks that allowed users to navigate and explore different documents and resources. This humble page laid the foundation for a revolutionary era of information sharing and transformed the way we access and interact with content on the internet.

The first version of HTML, called HTML 1.0, was released in 1993. It was a simple markup language with limited features compared to the modern versions we use today.

1.2.2 HTML Evolution

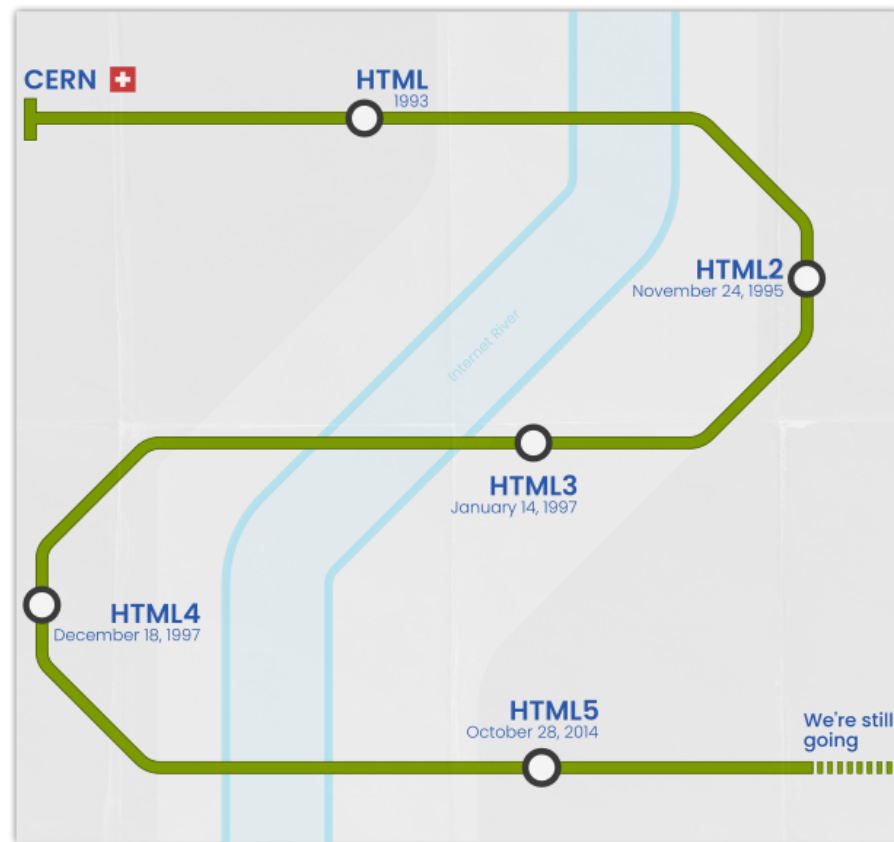
HTML has come a long way since its early days. Let's take a quick look at some important moments in its evolution:

Introduction of HTML1 (1993)

The first version of HTML that laid the foundation for web development. It introduced basic elements like headings, paragraphs, and lists, paving the way for the future of HTML.

Introduction of HTML2 (1995)

With HTML2, tables and the ability to embed images were introduced. This made it easier to create structured layouts and include visuals in web pages.



1.2.2 HTML Evolution

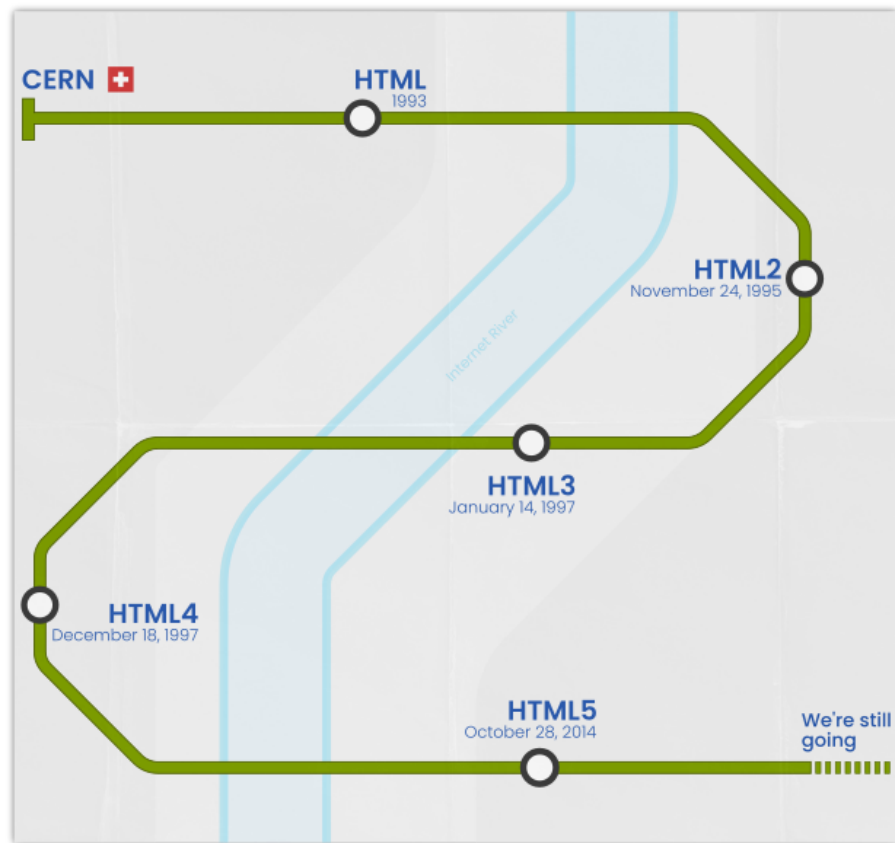
HTML has come a long way since its early days. Let's take a quick look at some important moments in its evolution:

HTML3 and HTML4 (1997)

HTML3 (1997) brought exciting features like frames, style sheets, and the ability to add interactivity with JavaScript. HTML4 (1997, republished in 1999) continued to build on those advancements and introduced support for multimedia elements.

HTML5 (2014)

HTML5 was a real game-changer! It introduced many new elements and features that make web development more powerful and versatile. With HTML5, we gained semantic elements for better structure, multimedia support for audio and video, and improved form handling.

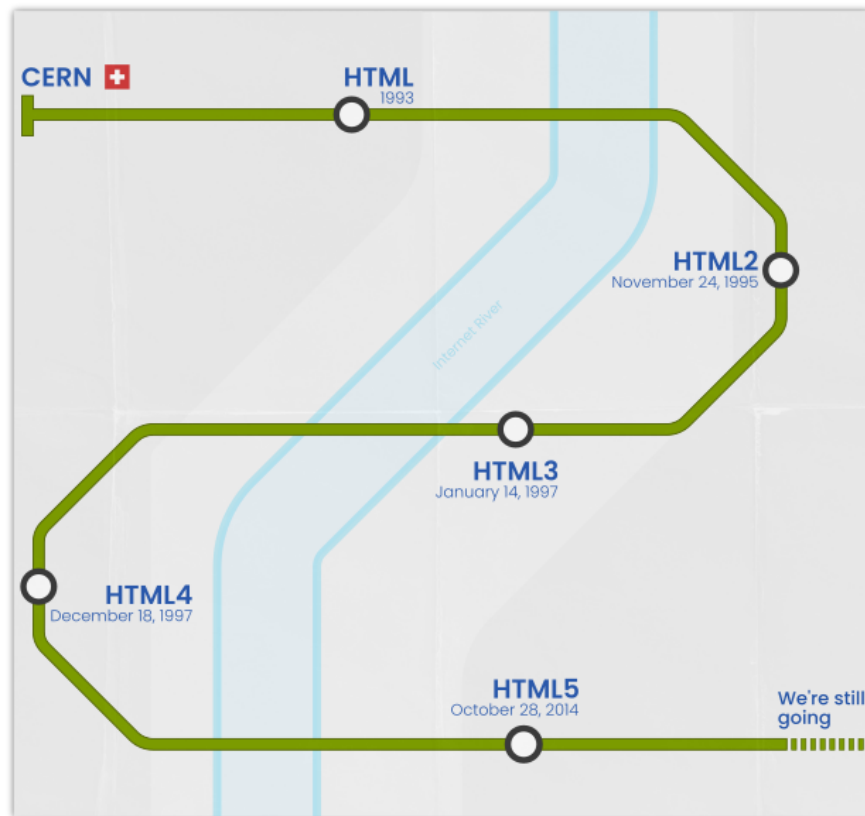


1.2.2 HTML Evolution

HTML has come a long way since its early days. Let's take a quick look at some important moments in its evolution:

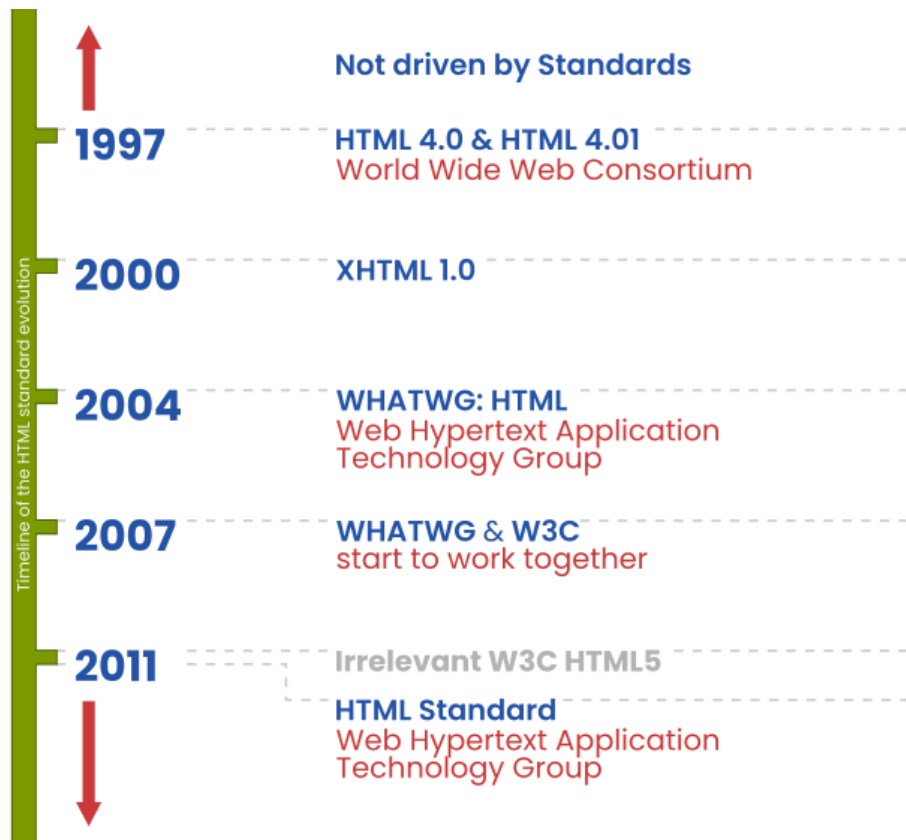
HTML5.1 And Beyond

HTML5.1 and subsequent versions continue to refine and enhance HTML, focusing on performance, security, and providing more tools for building modern web applications.



1.2.3 HTML Standards

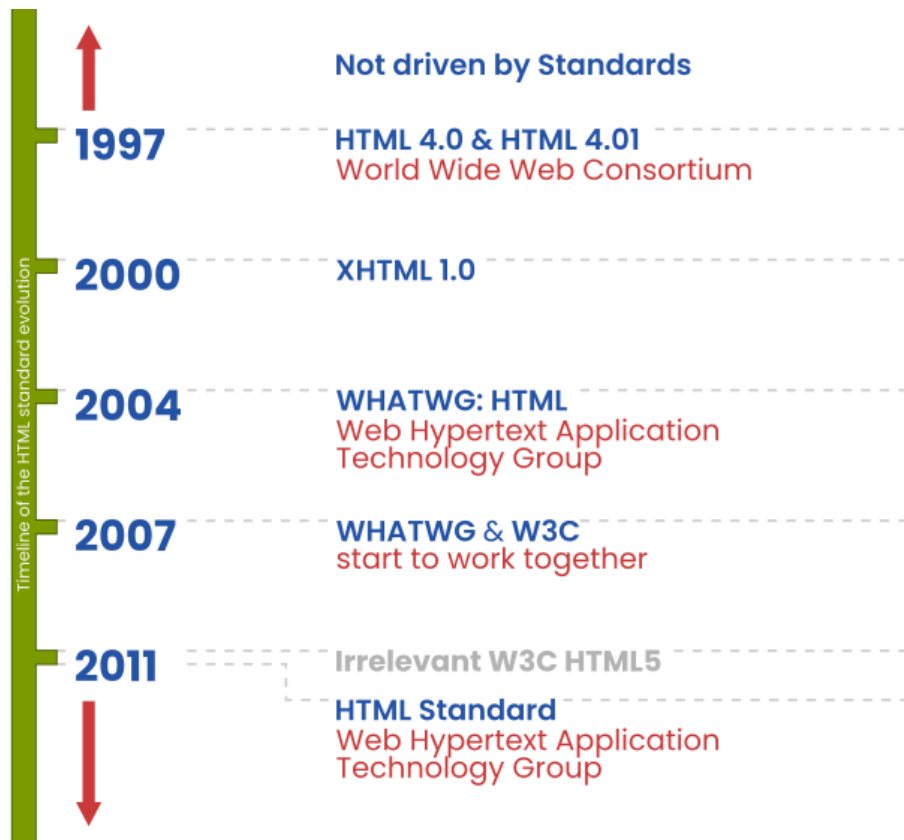
Did you know that behind the scenes, there are organizations dedicated to maintaining HTML standards and ensuring a smooth web experience for everyone? One such organization is the **World Wide Web Consortium, or W3C for short**. The other is the **Web Hypertext Application Technology Working Group (WHATWG)**.



1.2.3 HTML Standards

The W3C and WHATWG are international communities of web experts who develop and maintain the HTML standards. They work collaboratively to propose, review, and refine the specifications through a rigorous process of discussions, testing, and consensus-building. They aim to establish and uphold HTML standards that keep the Internet running smoothly and advance the capabilities of web development.

The main difference between W3C and WHATWG is their approach to HTML development, with W3C following a formal consensus-based process and WHATWG adopting a more pragmatic and agile approach.



1.2.4 The Future of HTML

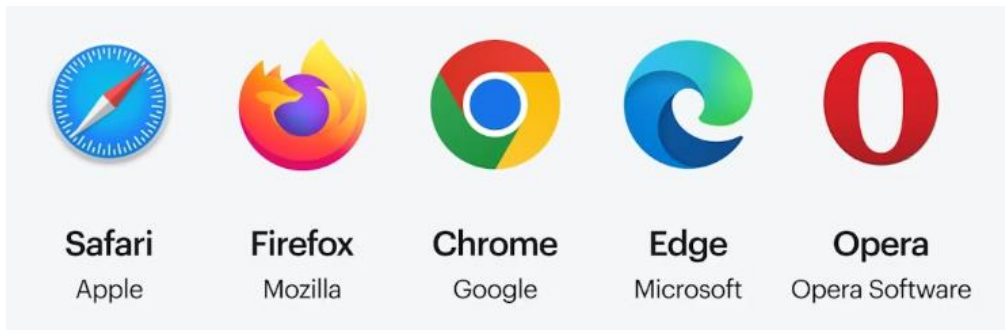
The future of HTML is an exciting realm where innovation and possibilities abound. As technology continues to advance, HTML is evolving to meet the demands of modern web development. One of the current trends in HTML is the focus on mobile-first design and responsiveness. With the increasing usage of mobile devices, web developers are embracing techniques like responsive web design, which allows websites to adapt and provide optimal user experiences on different screen sizes and devices. This trend emphasizes the importance of creating websites that are accessible and user-friendly across a wide range of devices, ensuring that users can access content seamlessly regardless of the device they are using.

Another emerging area in HTML is the integration of HTML with emerging technologies such as virtual reality (VR) and the Internet of Things (IoT). HTML is being enhanced to support the creation of immersive VR experiences directly within web browsers, making it easier for developers to create and deliver virtual reality content to users. Additionally, as IoT devices become more prevalent, HTML is being adapted to accommodate the unique requirements and interactions of IoT devices, enabling developers to create web-based applications that seamlessly integrate with smart devices.

1.3 HTML Development Environment

1.3.1 Setting Up An HTML Development Environment

To start writing and testing your HTML code, you'll need a minimum development environment that includes a couple of basic tools and resources. These tools include a text editor and a web browser.



Visual Studio Code



Sublime Text



Notepad++



1.3.2 Text Editor

A text editor is a software application that allows you to create and edit plain text files, and to write and edit code. For writing HTML code, you can use any basic text editor, such as Notepad on Windows or TextEdit on Mac.

However, using a dedicated text editor specifically designed for coding can greatly enhance your learning process. Text editors like:

- **Visual Studio Code,**
- **Sublime Text, and**
- **Notepad++**

Visual Studio Code



Sublime Text



Notepad++



1.3.3 Web Browser

A web browser is necessary to open, view and test your HTML code. Most modern web browsers include built-in developer tools that can help you debug your code and analyze your web page's performance. They are designed to interpret HTML code and display its content, formatting, and layout in a user-friendly way.

Examples of commonly used web browsers include:

Google Chrome



Mozilla Firefox



Microsoft Edge



1.3.4 Your First HTML File

Great! You're all set to create your first HTML file, and view your first web page. In order to get started, make sure you have a text editor and a web browser installed on your computer. These tools will be your best friends throughout this journey. Once you have them ready, follow these three steps:

Step 1 – Create an HTML file

Open your text editor and create a new file. It's time to give life to your web page! Save the file with a memorable name and make sure to use the .html extension, such as sample.html. This tells the computer that it's an HTML file.

Step 2 – Write the HTML code

Here comes the exciting part! Inside your HTML file, you'll start by adding the essential structure of an HTML document. We've got you covered with the code you need. Just copy the following:

1.3.4 Your First HTML File

index.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>My First Web Page</title>
5  </head>
6  <body>
7    <h1>Hello, World!</h1>
8    <p>This is my first web page.</p>
9  </body>
10 </html>
```

1.3.4 Your First HTML File

Great! You're all set to create your first HTML file, and view your first web page. In order to get started, make sure you have a text editor and a web browser installed on your computer. These tools will be your best friends throughout this journey. Once you have them ready, follow these three steps:

Step 3 – Save and open the HTML file

Now it's time to see your creation come to life! Save the HTML file and navigate to the location where you saved it. Double-click on the file, and like magic, it will open in your default web browser. Voila! You'll see your web page displayed beautifully.

Wow, you did it! You've successfully created and opened your first HTML file. Take a moment to appreciate your accomplishment and the incredible possibilities that lie ahead in your web development journey.

Remember, this is just the beginning. The more you practice and explore, the more you'll learn and discover. We hope you'll soon have fun experimenting with different HTML elements, adding images, and playing with styles to make your web page uniquely yours.

1.4 HTML Syntax: Elements, Tags, and Comments

1.5.1 What is HTML syntax?

In natural languages like English, Spanish, or French, syntax is about following the rules of grammar to put words in the right order and use punctuation correctly so that sentences make sense. It's like putting puzzle pieces together in a way that others can understand what you're trying to say.

Similarly, HTML syntax refers to the rules, guidelines, and conventions that define how HTML code should be structured and written. By following the correct syntax, you can ensure that your HTML code is properly understood by web browsers and displayed as intended.

HTML syntax defines the correct way to use HTML tags, attributes, and values (you'll learn what they are in a few moments) to create web pages that are both readable and functional by web browsers. Understanding it is essential for creating and maintaining effective and valid HTML code.

1.5.2 The DOCTYPE declaration

The DOCTYPE declaration is an integral part of HTML syntax and an important element in an HTML document¹ that helps web browsers understand how to interpret the document's content.

In simple terms, the DOCTYPE declaration tells the web browser which version of HTML is being used in the document. This is important because different versions of HTML have different rules and features. By including the DOCTYPE declaration, you are ensuring that the web browser knows which set of rules to follow when it displays the web page.

The DOCTYPE declaration is a special line of code that should appear at the very beginning of an HTML document, before any other content or elements. It consists of a string of characters² that identifies the version of HTML being used. The DOCTYPE declaration for HTML5 is `<!DOCTYPE html>`:

1.5.3 Elements and Tags

HTML is made up of a series of elements, which are defined by tags. The terms "elements" and "tags" are closely related but have distinct meanings.

HTML Elements

An HTML element is a part of a webpage, like a paragraph of text, an image, or a form. It's the basic unit of content and structure within an HTML document. Elements tell the web browser how to display the content.

HTML Tags

Tags are the markers that define the start and end of an element. They are written with angle brackets, like `<tag>` for the start of an element and `</tag>` (with a forward slash before the tag name) for the end. Tags label the type of content they enclose, instructing the browser on how to process it.

For instance, a paragraph is created with a `<p>` tag, indicating the start, and a `</p>` tag, marking the end, with the actual text content in between.

opening tag

`<p>`

content in between

content

closing tag

`</p>`

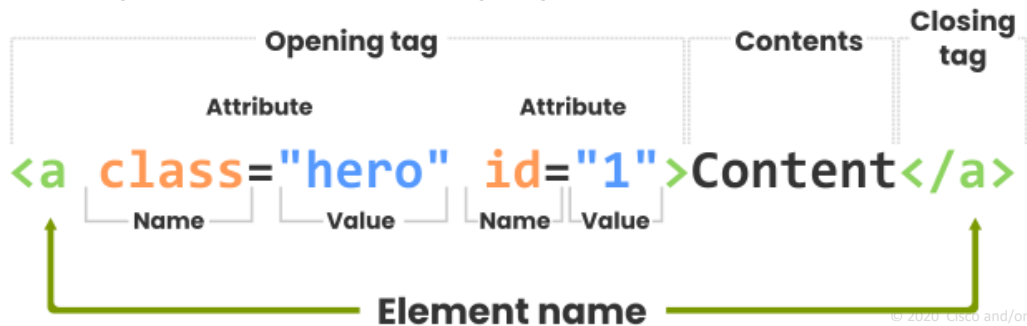
1.5.3 Elements and Tags

An HTML element encompasses everything from the start tag to the end tag, including the content and the tags themselves.

This means that the basic structure of an HTML element includes:

- the opening tag (<tag>),
- the content itself, and
- the closing tag (</tag>).

In essence, the opening tag marks the start of an element and can include attributes, which are extra details that configure or adjust the element's behavior. Attributes have a name and a value, separated by an equals sign, with the value enclosed in quotes. The content of the element is what's displayed or used, such as text or an image link, and the closing tag indicates the end of the element.



1.5.3 Elements and Tags

Self-Closing Tags

Some elements, like the image element (), don't contain content and therefore don't have a closing tag. These are known as self-closing tags.

For example:

index.html

```

```

1.5.3 Elements and Tags

Tags vs. Elements

In summary:

An element refers to the whole structure, including the opening tag, the content, and the closing tag (if present). It's what you see rendered on a webpage.

A tag refers specifically to the opening and closing markers that define the start and end of an element. They are not displayed on the webpage but instruct the browser on how to interpret the content.

HTML offers a variety of tags to organize and display content on the web. The number of these tags can vary with different versions of HTML, but HTML5 includes over 140 standard tags. Developers can also create custom elements to add new functionalities.

As HTML evolves, the list of available tags and elements changes, introducing new features and occasionally deprecating older ones. Staying updated with HTML standards is crucial for creating compatible and efficient web pages.

1.5.4 Block-Level and Inline Elements

In HTML5, there are two types of elements:

1. ***block-level elements, and***
2. ***inline elements.***

Block-Level Elements

Block-level elements are used to create the main structure of a web page, such as headers, paragraphs, and lists. They start on a new line and take up the full width of the page, which means that any content placed inside a block-level element will create a new block of content, separate from the content before or after it.

Examples of block-level elements include:

- `<h1>` through `<h6>` for headings,
- `<p>` for paragraphs,
- `<ul>` and `<ol>` for lists, and
- `<div>` for grouping content.

1.5.4 Block-Level and Inline Elements

Inline Elements

Inline elements, on the other hand, are used to style content within block-level elements, such as links and images. They don't start on a new line and only take up as much space as needed to display their content. This means that multiple inline elements can appear on the same line, separated by a space.

Examples of inline elements include:

- `<a>` for links,
- `<img>` for images, and
- `<span>` for styling content.

1.5.5 HTML Comments

Before we proceed further, let's introduce yet another important aspect of HTML syntax – comments.

HTML comments are a way to add notes or remarks in an HTML document that won't be visible to the user. Comments can be used to explain what certain parts of the code do or to temporarily disable parts of the code during development.

To create a comment, you simply need to enclose the text you want to comment out between `<!-- -->`.

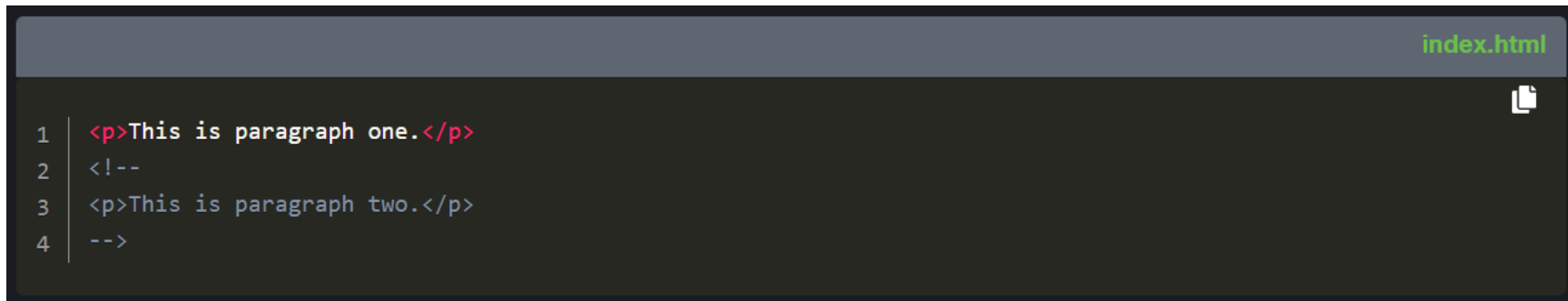
For example:

index.html

```
<!-- This is a comment. -->
```

1.5.5 HTML Comments

Now, let's try something out. What do you think will show up on the webpage? Go ahead, run the code to find out.

A screenshot of a code editor window. The title bar at the top right says "index.html" in green. The editor has a dark background. On the right side, there is a small icon of two overlapping sheets of paper. The code is as follows:

```
1 <p>This is paragraph one.</p>
2 <!--
3 <p>This is paragraph two.</p>
4 -->
```

In the example, you'll see that only the first paragraph `<p>This is paragraph one.</p>` appears on the webpage because it's not inside comment tags. But the second paragraph `<p>This is paragraph two.</p>` is inside comment tags, so it's hidden from view.

Comments are Visible in the Source Code

Even though comments don't show up on the webpage, they're still visible in the HTML code of the page. Keep in mind that search engines can also see HTML comments, so it's not a good idea to use them to hide information you don't want search engines or users to find.

1.6 HTML Syntax: Attributes and Values

1.6.1 Attributes and Values

Attributes provide additional information about an element, such as its style, source, or behavior. They are included within the opening tag and consist of a name-value pair.

You can add attributes to an HTML element by using the attribute name, followed by an equal sign and the attribute value. The value can be enclosed in quotes, single quotes, or left unquoted, depending on the specific attribute and the values it accepts.

For example, the `<img>` tag is used to insert an image in an HTML document. It has several attributes that can be used to specify the source of the image, its width and height, the text description for accessibility purposes, and other properties.

1.6.1 Attributes and Values

The most commonly used attributes for `<img>` are the `src` attribute, which specifies the path to the image file, and the `alt` attribute, which provides a text description of the image that is displayed when the image cannot be loaded or seen. Here's an example:

``.



1.6.2 Global vs. Element-Specific attributes

HTML attributes can be broadly classified into two categories:

- ***global attributes***, which can be applied to any HTML element. They are useful for a wide range of purposes, such as assigning unique identifiers, adding classes for styling, or providing additional information about an element. Examples include `id`, `class`, `style`, and `title`.
- ***element-specific attributes***, which are specific to particular HTML elements and cannot be used with all elements. They provide functionality or modify the behavior of specific elements, making them essential for proper usage of those elements. Examples include `href` (specific to `<a>` elements), `src` (specific to `<img>` and media elements), and `type` (specific to `<input>` elements).

Understanding the difference between global and element-specific attributes is crucial for using HTML effectively. While global attributes provide a consistent set of options for all elements, element-specific attributes allow you to tailor the behavior and appearance of individual elements to fit your needs.

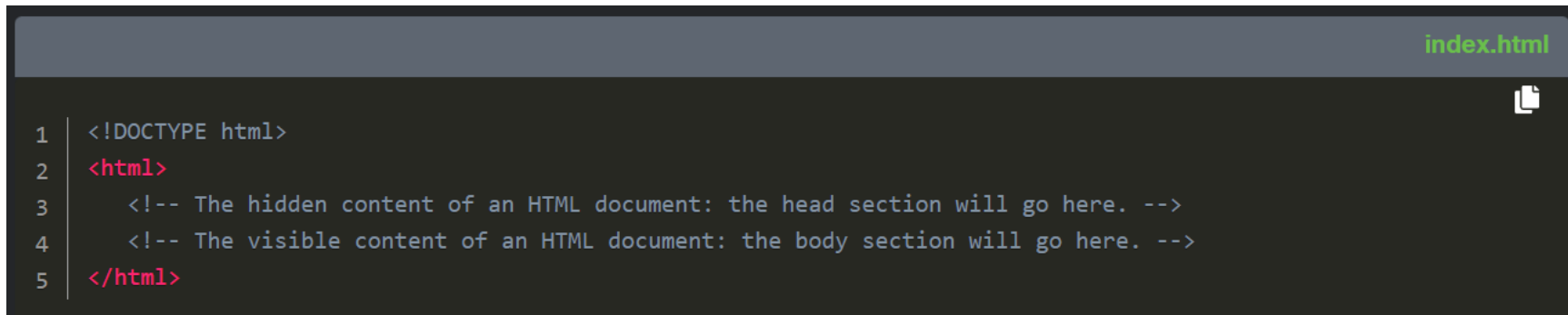
1.7 HTML Syntax: HTML, Head, and Body Tags

1.7.1 The HTML Tag

The HTML tag (<html>) is one of the most important tags in an HTML document, as it defines the entire web page and serves as the starting point for every HTML document. It informs the browser that the document is an HTML document, and it is the first tag in every HTML document.

The HTML tag is an opening tag that comes at the beginning of the document and a closing tag that comes at the end. Everything in the document appears between these two tags, and it does not have any attributes associated with it.

Here's an example of the HTML tag:



```
1 <!DOCTYPE html>
2 <html>
3   <!-- The hidden content of an HTML document: the head section will go here. -->
4   <!-- The visible content of an HTML document: the body section will go here. -->
5 </html>
```

1.7.2 The Head Section

The head section is an important component of an HTML document that contains information used by the browser to display and interact with the web page. It can optimize the web page for search engines and furnish additional context to users. The head section is hidden from the user's view and does not appear on the web page (it is a background portion of the document that operates behind the scenes).

Some of the key pieces of information that can be included in the head section are:

- Page title, i.e. the title of the web page that appears in the title bar of the browser and is used by search engines to display the page title in search results.

Example: `<title>My Web Page</title>`

- Metadata, which includes the description and keywords that are used by search engines to index and rank the web page, instructions for web crawlers how to index and display the web page, or information about the author and creation date of the page.

Example: `<meta name="author" content="Peter Jackson">`

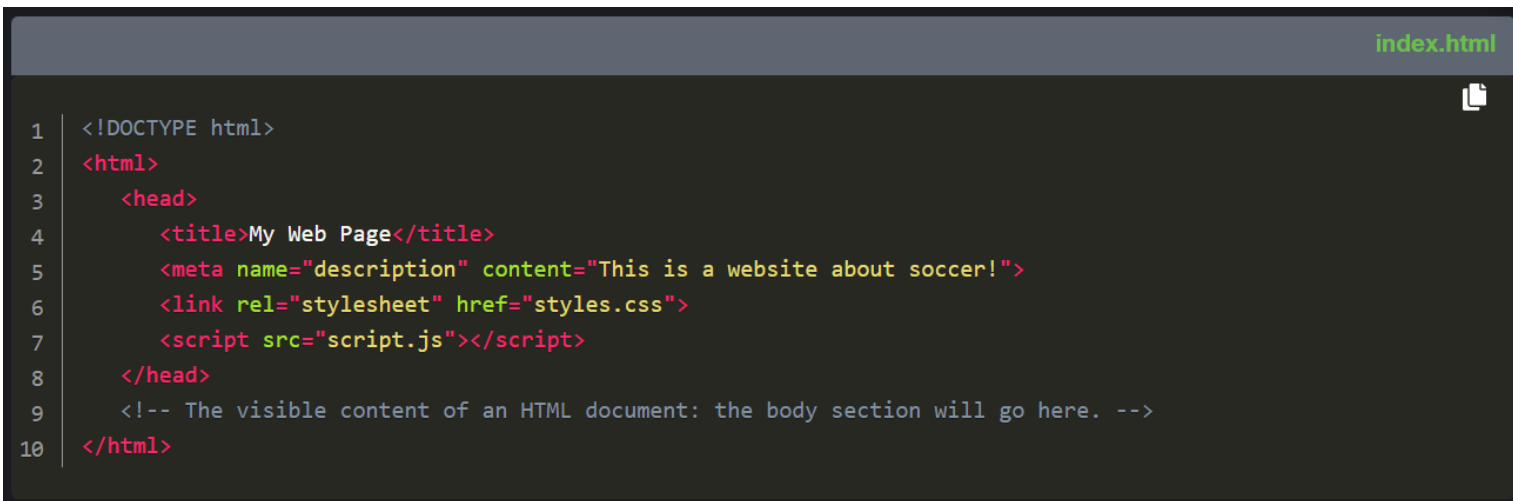
1.7.2 The Head Section

- Links to external files, which includes CSS stylesheets and JavaScript files that add styling and functionality to the web page.

Example: `<link rel="stylesheet" href="styles.css"> <script src="script.js"></script>`

- Character encoding, which tells the browser what character set to use when displaying the web page. (You already know that, don't you?)

Example: `<meta charset="UTF-8">`

A screenshot of a code editor with a dark background. The file name 'index.html' is shown in the top right corner. The code is written in a light-colored font and is numbered on the left side from 1 to 10. The code defines the basic structure of an HTML document, including the DOCTYPE declaration, the html root element, the head section with title, meta, link, and script tags, and the body section with a comment.

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>My Web Page</title>
5     <meta name="description" content="This is a website about soccer!">
6     <link rel="stylesheet" href="styles.css">
7     <script src="script.js"></script>
8   </head>
9   <!-- The visible content of an HTML document: the body section will go here. -->
10 </html>
```


1.7.3 The Body Section

The body section is the part of an HTML document that contains the visible content of the web page (i.e. text, images, videos, forms, links, etc.) and it's the area where you can get creative with your design and layout.

While the head section contains metadata and other information that is not visible on the web page, the body section contains the actual content that the user will see and interact with. The two sections work together in the sense that the head section provides instructions to the browser on how to display the content in the body section. Without the instructions provided by the head section, the browser may not be able to properly render the content in the body section.

1.7.3 The Body Section

Examples of content that you can include in the body section are:

- **Text:** You can use headings, paragraphs, and other text elements to present your content to the user.
- **Images:** You can include all types of images, such as photos, icons, and logos, to make your web page visually appealing.
- **Videos:** You can add video content, such as tutorials and product demos, to provide more information to the user. Videos are a great way to engage users and provide a more interactive experience.
- **Forms:** You can create elements that allow users to submit information, such as text boxes, dropdown menus, and buttons, to gather data from the user.
- **Links:** You can include links to other pages on the web or within the same website, making it easier for the user to navigate through your content.
- **Lists:** You can use lists to organize content, including unordered and ordered lists. Lists can be used to break up content into smaller, more manageable chunks, making it easier for users to read and understand.
- **Tables:** You can use tables to present data in a structured way, such as pricing information, product specifications, or schedules. Tables can be customized with colors, borders, and other visual elements to make them more engaging.

1.8 The Relationship Between HTML, CSS, and JS

1.8.1 Understanding the Interaction Between HTML, CSS, & JavaScript

HTML, CSS, and JavaScript are the three primary technologies used in web development to create dynamic and engaging web pages. Each technology has its own role to play in creating a fully functional and visually appealing web page:

- HTML provides the basic structure and content of the web page.
- CSS defines the presentation of HTML content, including its layout, colors, and fonts.
- JavaScript adds interactivity to the web page, making it more engaging and dynamic for the user.

To make things even simpler, if we were to describe each technology using just three words, it could look like this:

- HTML: Structure, content, markup
- CSS: Presentation, style, formatting
- JavaScript: Interactivity, functionality, behavior